



Introduction to AI Systems

BITS Pilani

Dr. Minati Rath
mrath@wilp.bits-Pilani.ac.in



<SS ZG662, Introduction to AI Systems >

Session 1

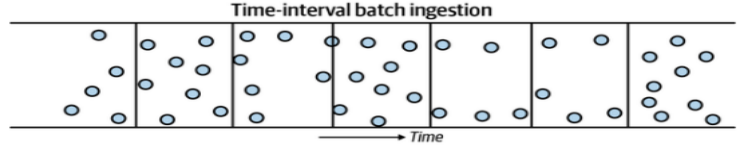
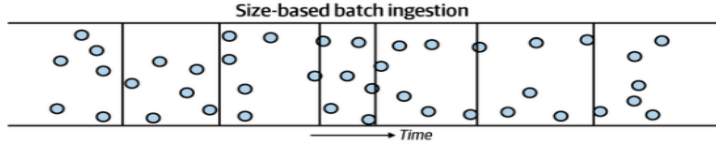
These slides are prepared by the instructor, with grateful acknowledgement of many others who made their course materials freely available online.



Data Engineering Pipelines, ETL/ELT, Batch and Streaming Pipelines, Data Lakes and Warehouses, Enterprise Data Pipeline Case Study



Batch Ingestion

- ~~Batch ingestion~~ - processing data in bulk, is often a convenient way to ingest data
 - data is ingested by taking a subset of data from a source system, based either on a time interval or the size of the data
- Time-interval batch ingestion
 - widespread in traditional business ETL for data warehousing
 - often used to process data once a day, overnight during off-hours, to provide daily reporting, but other frequencies can also be used
- Size-based batch ingestion
 - quite common when data is moved from a streaming-based system into object storage.
 - ultimately, must cut the data into discrete blocks for future processing in a data lake.
 - can break data into objects based on various criteria, such as the size in bytes or the total number of events.



- Commonly used batch ingestion patterns:
 - Snapshot or differential extraction
 - File-based export and ingestion
 - ETL versus ELT
 - Inserts, updates, and batch size, Data migration

Snapshot or Differential Extraction

Must choose whether to capture full snapshots of a source system or differential (incremental) updates

- With full snapshots, grab the entire current state of the source system on each update read
- With the differential update pattern, can pull only the updates and changes since the last read from the source system
- Differential updates are ideal for minimizing network traffic and target storage usage
- Full snapshot reads remain extremely common because of their simplicity.



File-Based Export and Ingestion

- Data is quite often moved between databases and systems using files
 - Data is serialized into files in an exchangeable format, and these files are provided to an ingestion system.
- A push-based ingestion pattern
 - data export and preparation work is done on the source system side
- File-based ingestion has several potential advantages over a direct database connection approach
 - often undesirable to allow direct access to backend systems for security reason
 - files can be provided to the target system in various ways
 - Common file-exchange methods are object storage, secure file transfer protocol (SFTP), electronic data interchange (EDI), or secure copy (SCP)



ETL Versus ELT

- Extremely common ingestion, storage, and transformation patterns encountered in batch workloads
- Extract
 - Getting data from a source system
 - Can be pulling data, or can be push based
 - Extraction may also require reading metadata and schema changes
- Load
 - Once data is extracted,
 - it can either be transformed (ETL) before loading it into a storage destination
 - or simply loaded into storage for future transformation (ELT)
 - should be mindful of the type of system you're loading, the schema of the data, and the performance impact of loading



Inserts, Updates, and Batch Size

- Batch-oriented systems often perform poorly when users attempt to perform many small-batch operations rather than a smaller number of large operations!
 - common to insert one row at a time in a transactional database, this is a bad pattern for many columnar databases
- Running many small in-place update operations is an even bigger problem
 - causes the database to scan each existing column file to run the update
- Need to understand the appropriate update patterns for the database or data store working with
- Certain technologies are purpose-built for high insert rates
 - Apache Druid and Apache Pinot can handle high insert rates
 - SingleStore can manage hybrid workloads that combine OLAP and OLTP characteristics
 - BigQuery performs poorly on a high rate of vanilla SQL single-row inserts but extremely well if data is fed in through its stream buffer



- **Migrating** data to a new database or environment is **not usually trivial**, and data needs to be moved in bulk
 - moving data sizes that are hundreds of terabytes or much larger, often involving the migration of specific tables and moving entire databases and systems
- As is often the case for data ingestion, **schema management** is a crucial consideration
 - **No matter how closely the two databases resemble each other, subtle differences almost always exist in the way they handle schema**
 - generally **easy to test ingestion of a sample of data** and find schema issues before undertaking a complete table migration
- Most data systems **perform best when data is moved in bulk rather than as individual rows or events**
 - File or object storage is often an excellent intermediate stage for



- Schema Evolution
 - common when handling event data - fields may be added or removed, or value types might change
 - can have unintended impacts on data pipelines and destinations - potentially impact downstream capabilities
- To alleviate issues
 - if event-processing framework has a schema registry, use it to version schema changes
 - A dead-letter queue can help investigate issues with events that are not properly handled
 - Low-fidelity route (and the most effective) is regularly communicating with upstream stakeholders about potential schema changes and proactively addressing schema changes with the teams introducing these changes instead of reacting to the receiving end of breaking changes.
- Late-Arriving Data
 - Event data might arrive late - common when ingesting data - because of latency issues
 - A group of events might occur around the same time frame (similar event times), but some might arrive later than others (late ingestion times)
 - should be aware of late arriving data and the impact on downstream systems and uses
 - To handle late-arriving data, need to set a cutoff time for when late-arriving data will no longer be processed.



- Ordering and Multiple Delivery
 - Streaming platforms are generally built out of **distributed systems**, which can cause some complications
 - Specifically, messages may be **delivered out of order** and more than once (**at-least-once delivery**)
- Replay
 - allows readers to **request a range of messages from the history**, allowing to rewind your event history to a particular point in time
 - a key capability in many streaming ingestion platforms and is particularly useful when **need to reingest and reprocess data for a specific time range**
 - For example,
 - **RabbitMQ typically deletes messages after all subscribers consume them**
 - **Kafka, Kinesis, and Pub/Sub all support event retention and replay**
- Message Size
 - easily **overlooked** issue - must ensure that the streaming framework in question can handle the maximum expected message size
 - Amazon Kinesis supports a maximum message size of 1 MB
 - Kafka defaults to this maximum size but can be configured for a maximum of 20 MB or more.



Time to Live(TTL)

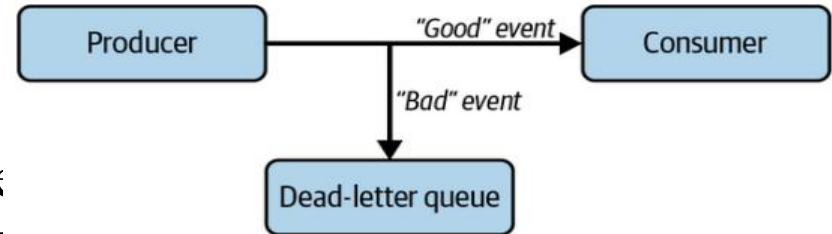
- **How long to preserve event record?**
- A key parameter is **maximum message retention time**, also known as the time to live (TTL)
 - usually **a configuration set** for how long want events to live before they are acknowledged and ingested
 - Any unacknowledged event that's not ingested after its TTL expires automatically disappears.
- Helpful to **reduce backpressure and unnecessary event volume** in event-ingestion pipeline.
 - An **extremely short TTL** (milliseconds or seconds) might cause most messages to **disappear** before processing
 - A **very long TTL** (several weeks or months) will create a **backlog of many unprocessed messages**, resulting in long wait times
- For example,
 - **Google Cloud Pub/Sub supports retention periods of up to 7 days**
 - **Amazon Kinesis Data Streams retention can be turned up to 365 days**
 - **Kafka can be configured for indefinite retention, limited by available disk space**

Error Handling and Dead-Letter Queues



Sometimes events aren't successfully ingested

- Perhaps
 - an event is sent to a nonexistent topic or message queue
 - the message size may be too large
 - or the event has expired past its TTL



Dead-letter queue

- Events that cannot be ingested **need to be rerouted and stored in a separate location** called a dead-letter queue
- **segregates problematic events from events that can be accepted by the consumer**
- If events are not rerouted to a dead-letter queue, these erroneous events **risk blocking other messages from being ingested**

Can be used to

- **diagnose** why event ingestion errors occur and solve data pipeline problems
- might be able to **reprocess some messages** in the queue after fixing the underlying cause of errors



- A consumer subscribing to a topic can get events in two ways: **push and pull**
- Kafka and Kinesis support only **pull subscriptions**
 - Subscribers read messages from a topic and confirm when they have been processed
- In addition to **pull subscriptions**, Pub/Sub and RabbitMQ support **push subscriptions**
 - allowing these services to write messages to a listener
- Pull subscriptions are the default choice for most data engineering applications,
 - but may want to consider push capabilities for specialized applications
 - **Pull-only message ingestion systems can still push if an extra layer is added to handle this**

Process of moving data from a source into a landing area or an object store where it can be used for ad hoc queries and analytics.

A simple data ingestion pipeline consumes data from a point of origin, cleans it up a bit, then writes it to a destination

Importance - Helps teams go fast!

Scope of any given data pipeline is **deliberately narrow**, giving data teams flexibility and agility at scale

Once parameters are set, **data analysts and data scientists** can easily build a single data pipeline to **move data to their system of choice**

Common examples of data ingestion include:

Move data from Salesforce.com to a data warehouse then analyze with Tableau

Capture data from a Twitter feed for real-time sentiment analysis

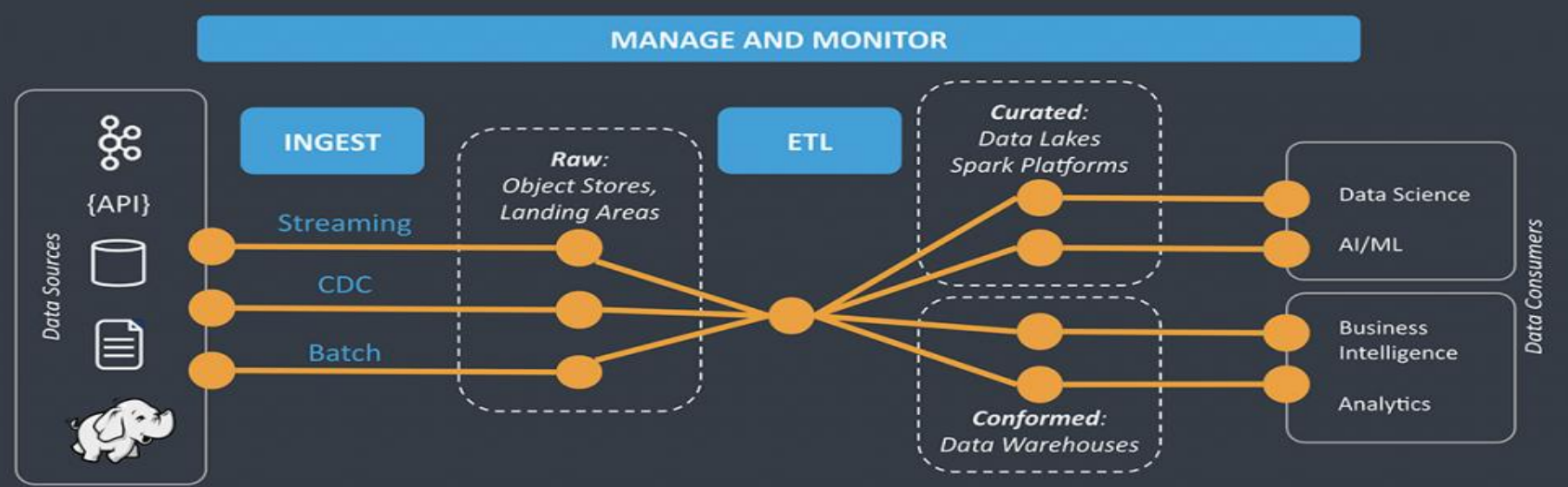
Acquire data for training machine learning models and experimentation

Modern Data Integration Begins with Data Ingestion

Data engineers use data ingestion pipelines to better **handle the scale and complexity of business demands for** data

Lots of intent-driven data pipelines operating continuously across the organization without direct involvement of a development team

Data ingestion has become a key component of self-service platforms for analysts and data scientists to access data for real-time analytics, machine learning and AI workloads.



- Data ingestion **extracts data from the source** where it was created or originally stored
 - Then **loads data into a destination or staging area**
- A simple data ingestion pipeline
 - might apply **one or more light transformations**
 - enriching or filtering data
 - before writing it to some set of destinations, a data store or a message queue
- More complex transformations
 - such as **joins, aggregates, and sorts**
 - for specific analytics, applications and reporting systems
 - can be done with **additional pipelines**

Data Sources

Data teams have moved way beyond the walled garden of the **enterprise data center** for insight increasingly load data from across **business units as well as 3rd party and unstructured data**, start data loads, when and where they need it

Common data source types:

Apache Kafka, JDBC, Oracle CDC

HTTP Clients, HDFS

Data Destinations

A data ingestion pipeline may simply send data to an **application or messaging system**, or store **ingested data in a data lake or cloud object storage** for use in relational and NoSQL databases or **data warehouses**

Common destination:

Apache Kafka , JDBC

Snowflake, Amazon S3, Databricks

Cloud Data Migration

Enterprise business processes are **moving to cloud-based platforms** for storing, processing and **applications**

Data ingestion workloads have become essential **to cloud migration**

But moving data out of silos and into agile cloud data lakes or powerful cloud data warehouses, generates **some uncomfortable questions**

The more data platforms can automate and operationalize the what-ifs of data ingestion, the better they can support the growing demand for continuous, reliable data.

Data Ingestion vs Data Integration



Data ingestion **originated as a small part of data integration**, a more complex process required to make data consumable in new systems before loading it

Data integration usually **requires advance specification from source to schema to transformation to destination**

With data ingestion, a few light transformations may be made, such as masking personally identifiable information (PII)

but most of the work depends on the end use and happens after landing the data

Think of it this way:

Data integration includes **processes to prepare data to be consumable at its final destination**

Data ingestion **gets data to places where preparation happens in response to downstream needs**

Data ingestion works well for streaming data that can be used immediately with very few transformations or as a way to collect data for ad hoc analysis

By focusing on the ingestion part of the data lifecycle, companies have been able to speed up the availability of data for innovation and growth

Data Ingestion Challenges

- ~~With the rise of big data, cloud computing, and the demand for real-time analytics~~, the capacity for data increased dramatically
 - old ETL processes began to ~~slow data teams~~ down compared to the ELT model
- Complexity Takes Time
 - The to-do list for data engineering is long and getting longer
 - Building data pipelines from scratch every time a new source or business need comes up ~~slows down the whole data team~~
- Change Takes Time
 - Every ~~change~~ or evolution of a target system ~~generates 10-20 hours of work for a data engineer~~
 - 90% of time will be spent on ~~maintenance and break-fix, changes that are considered data drift~~
- Maintenance and Rework Takes Time
 - Doing the ~~same thing over and over~~ with a lot of troubleshooting and debugging doesn't leave much time for ~~innovation or ramping up on new technologies~~



Types of Data Ingestion Tools

Manual/Hand Coding

One way to ingest data may be to hand code a data pipeline, assuming know how to code and are familiar with the **languages** needed

gives the greatest control

may spend a lot of time **working and reworking code**

Single-purpose Tools

Basic data ingestion tools provide a **drag-and-drop interface with lots of pre-built connectors and transformation**

a **quick way** to get a lot done or to enable less skilled data consumers,
management of many drag-and-drop data pipelines will be **cumbersome**

can't share work with team or the analysts and data scientists knocking on door

Data Integration Platforms

Traditional data integration platforms **incorporate features for every step of the data value chain**

means most likely **need developers and architectures specific to each domain**

making it **difficult to move fast** and adapt easily to change

A DataOps Approach

Applying **agile methodologies to data**, a DataOps approach to data pipelines **automates as much as possible** and **abstracts away the “how” of implementation**

Data engineers can focus on the **“what” of the data**, and responding to business needs

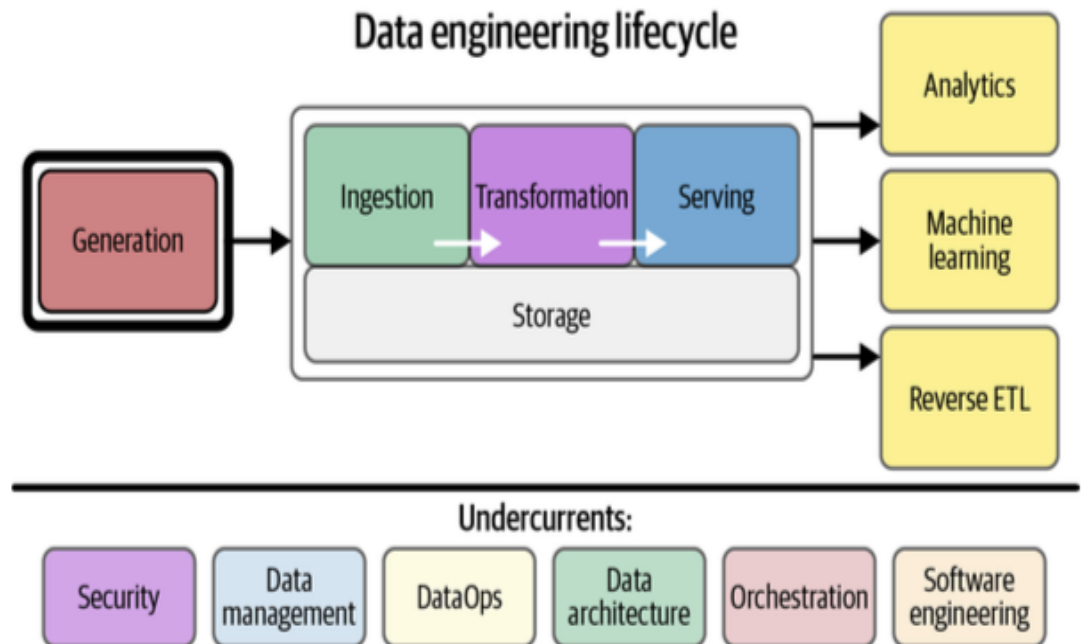
Primary considerations related to data ingestion

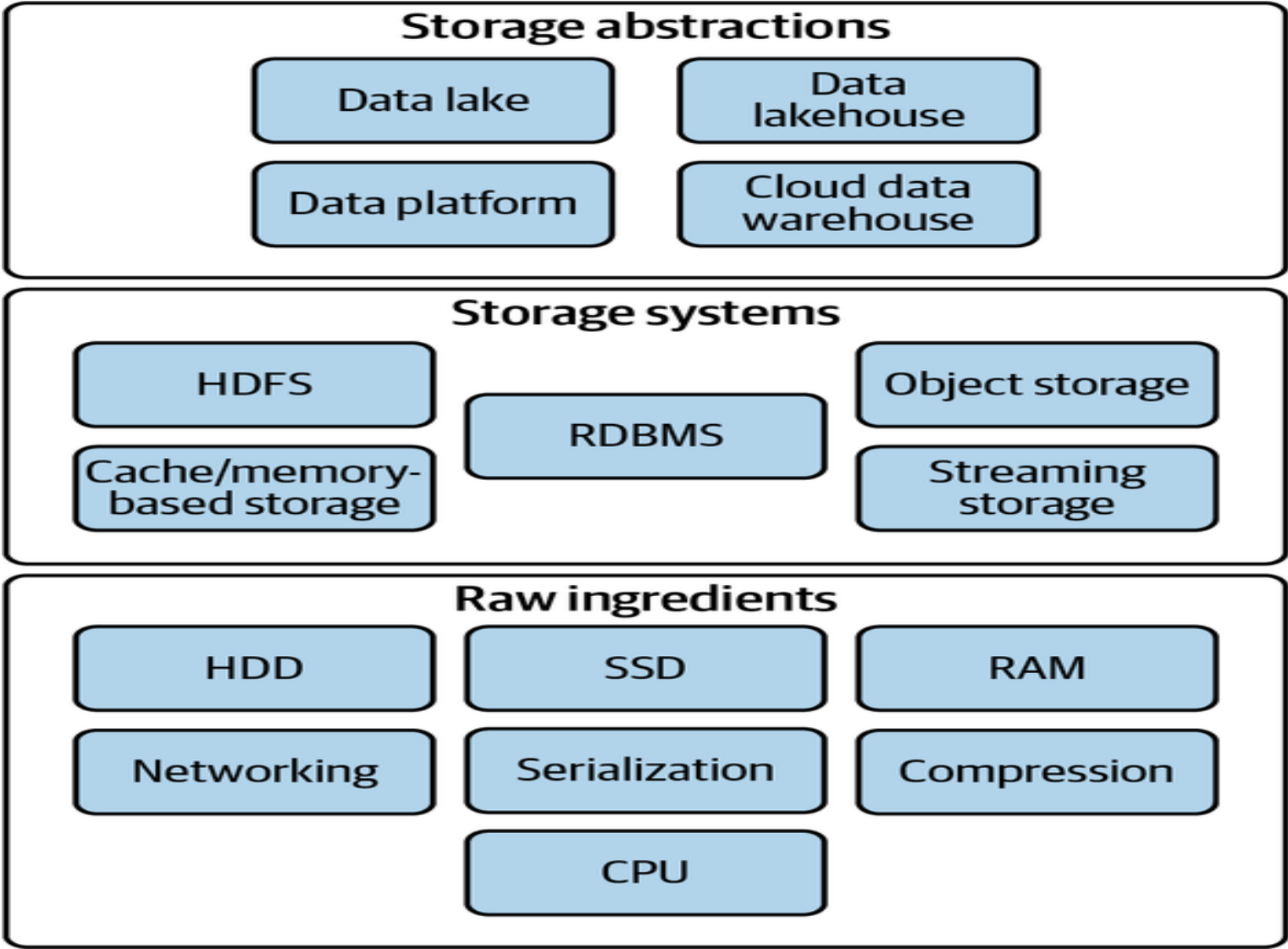


- What's the use case for the data is ingesting?
- Can this data be reused and avoid ingesting multiple versions of the same dataset?
- Where is the data going? What's the destination? – Lineage
- How often should the data be updated from the source? - Freshness
- What is the expected data volume?
- What format is the data in? Can downstream storage and transformation accept this format?
- Is the source data in good shape for immediate downstream use? Is the data of good quality?
- What post-processing is required to serve it? What are data-quality risks?
- Does the data require in-flight processing for downstream ingestion if the data is from a streaming source?

Cornerstone in the data engineering life cycle

- Underlies major stages – ingestion, transformation and serving
- Get stored many times when it moved through lifecycle







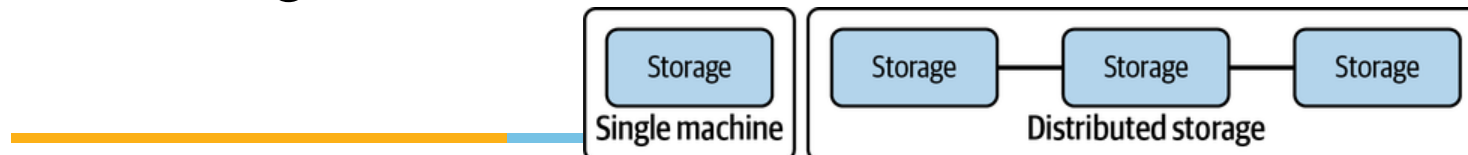
- Hardware
 - Magnetic Disk Drive
 - Solid-State Drive
 - Random Access Memory
 - Networking and CPU
- Associated features
 - Serialization
 - Compression
 - Caching

Table 6-1. A heuristic cache hierarchy displaying storage types with approximate pricing and performance characteristics

Storage type	Data fetch latency ^a	Bandwidth	Price
CPU cache	1 nanosecond	1 TB/s	N/A
RAM	0.1 microseconds	100 GB/s	\$10/GB
SSD	0.1 milliseconds	4 GB/s	\$0.20/GB
HDD	4 milliseconds	300 MB/s	\$0.03/GB
Object storage	100 milliseconds	10 GB/s	\$0.02/GB per month
Archival storage	12 hours	Same as object storage once data is available	\$0.004/GB per month



- As data storage and access patterns become more complex and outgrow the usefulness of a single server
 - distributing data to more than one server becomes necessary
- Data can be stored on multiple servers, known as distributed storage
 - with purpose is to store data in a distributed fashion
- Distributed storage coordinates the activities of multiple servers
 - to store, retrieve, and process data faster and at a larger scale, all while providing redundancy in case a server becomes unavailable
 - provides built-in redundancy and scalability for large amounts of data
- Examples, **object storage, Apache Spark, and cloud data warehouses** rely on distributed storage architectures





- A file is a data entity with specific read, write, and reference characteristics used by software and operating systems
- A file
 - has finite length - stream of bytes
 - supports append operations - append bytes to the files
 - supports random access - read from any location in the file or write updates to any location
- File storage systems organize files into a directory tree
 - /Users/Pravin/file.txt
- The filesystem stores each directory as metadata about the files and directories that it contains
 - consists of the name of each entity, relevant permission details, and a pointer to the actual entity
 - To find a file on disk, the operating system looks at the metadata at each hierarchy level and follows the pointer to the next subdirectory entity until finally reaching the file itself.

Data Storage Systems - File Storage - Types



Local disk storage

The most familiar type of file storage is an operating system–managed filesystem on a local disk partition of SSD or magnetic disc

New Technology File System (NTFS) and ext4 are popular filesystems on Windows and Linux respectively
OS handles the details of storing directory entities, files, and metadata

designed to write data to allow for easy recovery in the event of power loss during a write, though any unwritten data will still be lost.

support full read after write consistency; reading immediately after a write will return the written data

employs various locking strategies to manage concurrent writing attempts to a file

Network-attached storage

provide a file storage system to clients over a network

prevalent solution for servers; they quite often ship with built-in dedicated NAS interface hardware

has performance penalties to accessing the filesystem over a network

significant advantages includes redundancy and reliability, fine-grained control of resources and file sharing across multiple machines

Cloud filesystem services

provide a fully managed filesystem for use with multiple cloud VMs and applications, potentially including clients outside the cloud environment

should not be confused with standard storage attached to VMs—generally, block storage with a filesystem managed by the VM operating system

behave much like NAS solutions, but the details of networking, managing disk clusters, failures, and configuration are fully handled by the cloud vendor

Amazon Elastic File System (EFS) is an extremely popular example of a cloud filesystem service



- Fundamentally type of **raw storage provided by SSDs and magnetic disks**
- In the cloud, virtualized block storage is the **standard for VMs**
 - allow fine control of storage size, scalability, and data durability beyond that offered by raw disks.
- A block is the smallest addressable unit of data supported by a disk
 - was often **512 bytes** of usable data on older disks, but it has now grown to **4,096 bytes** for most current disks
 - also contain **extra bits for error detection/correction and other metadata**
- Blocks on magnetic disks are geometrically arranged on a physical platter
 - Two blocks on the same track can be read without moving the head, while reading two blocks on separate tracks requires a seek
 - Seek time can occur between blocks on an SSD, but this is infinitesimal compared to the seek time for magnetic disk tracks
- Block storage applications
 - Transactional database systems generally access disks at a block level to lay out data for optimal performance
 - Operating system boot disks on cloud VMs



RAID

- redundant array of independent disks
- simultaneously controls multiple disks to
 - improve data durability
 - enhance performance
 - combine capacity from multiple drives
- an array can appear to the operating system as a single block device

Storage area network (SAN)

- provide virtualized block storage devices over a network, typically from a storage pool
- allow fine-grained storage scaling and enhance performance, availability, and durability
- common with on-premises storage systems

Data Storage Systems - Block Storage - Types



- ~~Cloud virtualized block storage~~
 - similar to SAN but free engineers from dealing with SAN clusters and networking details
 - Example, **Amazon Elastic Block Store (EBS)** - default storage for Amazon EC2 virtual machines
 - store data separate from the instance host server but in the same zone to support high performance and low latency
 - allows EBS volumes to persist when an EC2 instance shuts down, when a host server fails, or even when the instance is deleted
 - suitable for applications such as databases, where data durability is a high priority
 - replicates all data to at least two separate host machines, protecting data if a disk fails
- Local instance volumes
 - Cloud providers also offer block storage volumes that are physically attached to the host server running a virtual machine
 - generally very low cost (included with the price of the VM in the case of **Amazon's EC2 instance store**) and provide low latency
 - when a VM shuts down or is deleted, the contents of the locally attached disk are lost, whether or not this event was caused by intentional user action
 - ensures that a new virtual machine cannot read disk contents belonging to a different customer.



Object Storage

- contains objects of all shapes and sizes - a specialized file-like construct —TXT, CSV, JSON, images, videos, or audio
- have grown in importance and popularity with the rise of big data and the cloud
- Amazon S3, Azure Blob Storage, and Google Cloud Storage (GCS) are widely used
- many cloud data warehouses (and a growing number of databases) utilize object storage as their storage layer
- cloud data lakes generally sit on object stores

Object storage was arguably one of the first “serverless” services

- engineers don’t need to consider the characteristics of underlying server clusters or disks

An object store is a key-value store for immutable data objects.

- don’t support random writes or append operations; instead, they are written once as a stream of bytes
- After this initial write, objects become immutable
- To change data in an object or append data to it, must rewrite the full object



- Object Storage
 - contains objects of all shapes and sizes - a specialized file-like construct — TXT, CSV, JSON, images, videos, or audio
 - have grown in importance and popularity with the rise of big data and the cloud
 - Amazon S3, Azure Blob Storage, and Google Cloud Storage (GCS) are widely used
 - many cloud data warehouses (and a growing number of databases) utilize object storage as their storage layer
 - cloud data lakes generally sit on object stores
- Object storage was arguably one of the first “serverless” services
 - engineers don’t need to consider the characteristics of underlying server clusters or disks
- An object store is a key-value store for immutable data objects.
 - don’t support random writes or append operations; instead, they are written once as a stream of bytes
 - After this initial write, objects become immutable
 - To change data in an object or append data to it, must rewrite the full object



- RAM offers excellent latency and transfer speeds but extremely vulnerable to data loss
- RAM-based storage systems are generally focused on caching applications, presenting data for quick access and high bandwidth
 - Data should generally be written to a more durable medium for retention purposes
 - These ultra-fast cache systems are useful when data engineers need to serve data with ultra-fast retrieval latency
- Memcached and lightweight object caching
 - a key-value store designed for caching database query results, API call responses, and more
 - uses simple data structures, supporting either string or integer types
 - deliver results with very low latency while also taking the load off backend systems
- Redis, memory caching with optional persistence
 - a key-value store, but it supports somewhat more complex data types (such as lists or sets)
 - with a typical configuration, Redis writes data roughly every two seconds
 - Redis is thus suitable for extremely high-performance applications but can tolerate a small amount of data loss.



- HDFS is
 - based on Google File System (GFS) and was initially engineered to process data with the MapReduce programming model
 - similar to object storage but with a key difference: Hadoop combines compute and storage on the same nodes, where object stores typically have limited support for internal processing
- Management
 - breaks large files into blocks, chunks of data less than a few hundred megabytes in size
 - managed by the NameNode, which maintains directories, file metadata, and a detailed catalog describing the location of file blocks in the cluster
 - each block of data is replicated to three nodes - increases both the durability and availability of data
- Claims that Hadoop is dead! This is only partially true!
 - no longer a hot, bleeding-edge technology
 - Many Hadoop ecosystem tools such as Apache Pig are now on life support and primarily used to run legacy jobs
 - The pure MapReduce programming model has fallen by the wayside
- but HDFS remains widely used in various applications and organizations
 - HDFS is a key ingredient of many current big data engines, such as Amazon EMR
 - In fact, Apache Spark is still commonly run on HDFS clusters



- Streaming data has different storage requirements than nonstreaming data - **retention, replay!**
- Retention
 - In the case of message queues, stored data is temporal and expected to disappear after a certain duration
 - Distributed, scalable streaming frameworks like Apache Kafka now allow extremely long-duration streaming data retention
 - Kafka competitors (including Amazon Kinesis, Apache Pulsar, and Google Cloud Pub/Sub) also support long data retention
- Replay
 - allows a streaming system to return a range of historical stored data
 - standard data-retrieval mechanism for streaming storage systems
 - can be used to run batch queries over a time range or to reprocess data in a streaming pipeline.



- Data engineering storage abstractions are **data organization and query patterns** that sit at the heart of the data engineering lifecycle
 - are built atop the data storage systems
- The main types of abstractions are those that support **data science, analytics, and reporting** use cases
 - Data warehouse
 - Data lake
 - Data lakehouse
 - Data platforms



- Data Validation
 - means checking the accuracy and quality of source data before training a new model version
 - ensures that anomalies that are infrequent or manifested in incremental data are not silently ignored
 - focuses on checking that the statistics of the new data are as expected (e.g. feature distribution, number of categories, etc)
- Examples of such validations in the machine learning pipeline:
 - Are there any **anomalies or data errors** in the incremental data?
 - Are there any **assumptions on data** that are taken **during model training** and are getting violated during serving?
 - Are there **significant differences between training and serving data**?
 - Are there differences between successive data that are getting added into training data?
- Output from the data validation steps should be **informative** enough for a data engineer to take action
 - needs to have **high precision** as too many false alarms will easily be lost credibility



- Creating data validation rules for a dataset with few columns does sound simple
 - when the number of columns in datasets increases, it becomes a humongous task
- Tracking and comparing metrics from the historical datasets to find anomalies in historical trends for each column needs a good amount of recurring time from a data scientist
- Today applications are expected to run 24-by-7 and in such scenarios, data validation needs to be automated
 - data validation component should be smart enough to refresh validation rules

Working of data validation component



- Data validation component serves as a guard post of the ML application
 - does not let bad quality data in
 - keeps a check on each and every new data entry that is going to add to the training data



Data Validation Workflow

- Steps
 - Calculate the statistics from the training data against a set of rules
 - Calculate the statistics of the ingested data that has to be validated
 - Compare the statistics of the validation data with the statistics from the training data
 - Store the validation results and takes automated actions like removing the row, capping, or flooring the values
 - Sending the notification and alerts for approval



Unit-test approach by Amazon Research - Deeque

- In software engineering, engineers write unit tests to test their code
 - Similarly, unit tests should also be defined to test the incoming data
- AWS Deeque framework follows the below principles
- Declare constraints:
 - User defines how their data should look like
 - by declaring checks on their data by composing constraints on various columns
- Compute metrics
 - Based on the declared constraint, translate them to measurable metrics
 - metrics can be computed and compared between the data in hand and the incremental data
- Analyze and report
 - Based on the collected metrics over time, predict if the metric on the incremental data is an anomaly or not
 - As a rule, the user can have the system issue “a warning” if the new metric is more than three standard deviations away from the previous mean or throw “an error” if it is more than four standard deviations away.
- Basis the analysis, report the constraints that fail including the value(s) that made the constraint fail.

Deque - Constraints

Table 1: Constraints available for composing user-defined data quality checks.

constraint	arguments	semantic
<i>dimension completeness</i>		
isComplete	column	check that there are no missing values in a column
hasCompleteness	column, udf	custom validation of the fraction of missing values in a column
<i>dimension consistency</i>		
isUnique	column	check that there are no duplicates in a column
hasUniqueness	column, udf	custom validation of the unique value ratio in a column
hasDistinctness	column, udf	custom validation of the unique row ratio in a column
isInRange	column, value range	validation of the fraction of values that are in a valid range
<i>statistics (can be used to verify dimension consistency)</i>		
hasSize	udf	custom validation of the number of records
hasTypeConsistency	column, udf	custom validation of the maximum fraction of values of the same data type
hasCountDistinct	column	custom validation of the number of distinct non-null values in a column
hasApproxCountDistinct	column, udf	custom validation of the approx. number of distinct non-null values
hasMin	column, udf	custom validation of a column's minimum value
<i>time</i>		
hasNoAnomalies	metric, detector	validation of anomalies in time series of metric values

Table 2: Computable metrics to base constraints on.

metric	semantic
<i>dimension completeness</i>	
Completeness	fraction of non-missing values in a column
<i>dimension consistency</i>	
Size	number of records
Compliance	ratio of columns matching predicate
Uniqueness	unique value ratio in a column
Distinctness	unique row ratio in a column
<i>statistics (can be used to verify dimension consistency)</i>	
Minimum	minimal value in a column
Maximum	maximal value in a column
Mean	mean value in a column
StandardDeviation	standard deviation of the value distribution in a column

Data schema approach for data validation by Google Research



Tensorflow data validation

Google Research has come up with a very similar technique

- adopted “battle-tested” principles from the data management system and customized it for ML
- first codifies the expectation from correct data and using these expected statistics along with user-defined validation schema performs data validation

• Components

Data Analyzer

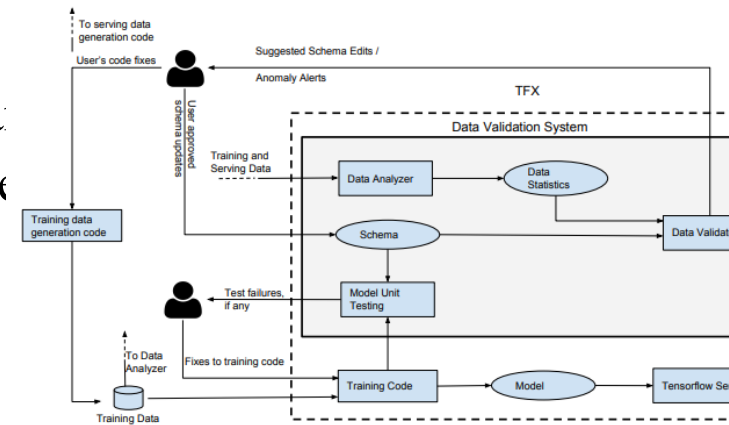
- computes a predefined set of data statistics required to define data

Data Validator

- checks for properties of data as specified through a schema
- schema is a precursor for a data validator to perform
- schema list out all the constraints on the features for basic checks and ML-related checks

Model Unit Tester

- checks for errors in training code using synthetic data generated through the schema



Deequ (Amazon)

1 No visualization available

2 Recalculates training statistics by aggregating prior saved training statistics and new data statistics.

3 Provides capability to do anomaly detection based on running average and standard deviation in addition to the threshold or absolute/relative difference from training data.

4 Supports data only in SparkDataFrame.

Tensorflow data validation (Google)

Provides visualization using Google Facets. It summarises statistics for each feature and compares the training and validation data.

Calculates statistics on whole training data in every run unless specified. This may become computationally expensive.

Provides capability to do anomaly detection based on the threshold or absolute/relative difference from training data.

Supports pandas dataframe, CSV, and best works with TFRecord.

Differences between Data Validation libraries



- Machine learning (ML) in production is quite different from ML in a Kaggle competition
 - One of the biggest difference is in the **data sets**
- Kaggle data sets are always **static**
 - given a fixed data set for training and you will be evaluated on a fixed data during testing/evaluation.
- Production data tends to be **constantly changing** over different dimension
 - i.e. time-wise and system-wise
- **Monitoring the predictive performance of an ML model in production has emerged as a crucial area of MLOps**
- Two common causes of decay in a model's predictive performance over time are the following:
 - Data Drfit
 - Concept drift



- Data drift
 - A skew grows between training data and serving data
- In data drift, the production data that a model receives for scoring has diverged from the dataset that was used to train, tune, and evaluate the model
- The discrepancies between training data and serving data can usually be classified as
 - Schema skew
 - Distribution skew



- Schema skew occurs when training data and serving data don't conform to the same schema
 - often caused by faults or changes in the upstream process that generates the serving data
- Schema deviations between training and serving data can include the following:
- Inconsistent features
 - a new feature is added to the serving data
- Inconsistent feature types
 - a numerical feature that was an integer number in training data is a real number in serving data
- Inconsistent feature domains
 - a value in a categorical feature disappears, or there's a change in the range of numerical features



- Distribution skew occurs when the distribution of feature values for training data is significantly different from serving data
 - can be the result of choosing the wrong training dataset to represent real-world data
 - can also happen naturally as new trends and patterns emerge in the data due to the changes in the dynamics of the environment
- Examples include
 - changes in the prices of real estate
 - a change in the popularity of fashion items



- Concept drift means that interpretation of the data has changed
 - As interpretation of the relationship between the input predictors and the target feature evolves
- Often, implies that
 - the mapping of input features to labels that are used during training is no longer valid,
 - or that a novel class or a range of label values has appeared
- Often the result of a change in the process you're attempting to model
 - can also be an evolution of your understanding of this process



- Prerequisites
 - The system needs to log all incoming features and predictions of the request and response
 - Log the ground truth as well of the associated request, which can be used as additional training data (this is for detecting label drift)
- Use statistical methods to actually detect it
- Supervised monitoring
 - Statistical Process Control
 - Sequential Analysis
 - Adaptive windowing
- Unsupervised monitoring
 - Clustering/novelty detection
 - Feature distribution monitoring
- Model dependent monitoring
 - MD3 – Margin Density Drift Detection

[Source: Production ML: Skews, Data Drift, and Concept Drift](#)



“Past performance is no guarantee of future results.”

- Model decay or drift
 - ML models in production can experience reduced performance over time
 - not only due to being fed bad data and poor programming
 - but also due to datasets and profiles that are constantly evolving
- Natural occurrence in ML models and the speed of decay can vary greatly
 - In some models, it can take years
 - In others, it can happen over the course of a few days
- One of the biggest post-production problems that can lead to an expedited rate of decay is **data-serving skew**
 - a problem that can arise quite easily and be difficult to detect

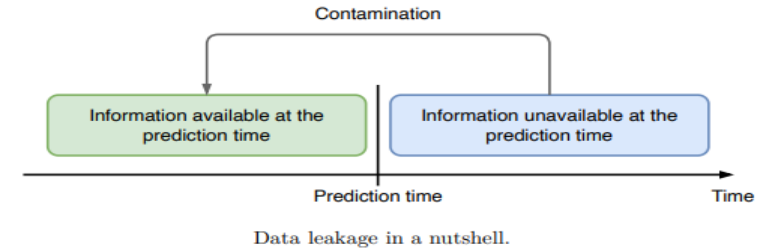
[What is training-serving skew in Machine Learning?](#)



- Training-serving skew is a difference between ML model outputs during the training and during serving (deployment)
 - essentially a discrepancy between an ML model's feature engineering code during training and during deployment
- Can be caused by:
 - A discrepancy between how data is handled in the training and serving pipelines
 - A change in the data between training and serving
 - A feedback loop between model and algorithm
- Very easy for training-serving skew to crop up
 - via a bug in model's code and cause serious repercussions further down the line, potentially stopping model from working entirely

[What is training-serving skew in Machine Learning?](#)

- Data leakage, also called target leakage - a problem affecting several stages of the machine learning life cycle from data collection to model evaluation



- Data leakage in supervised learning is the unintentional introduction of information about the target that should not be made available
 - also called as “contamination”
 - Training on contaminated data leads to overly optimistic expectations about the model performance



- Three most frequent causes of data leakage that can happen during data collection and preparation:
 - target being a function of a feature,
 - feature hiding the target,
 - feature coming from the future



Target being a function of a feature

- Let goal be to predict a country's GDP based on various attributes:
 - area, population, geographic region, and so on
 - Gross Domestic Product (GDP) is defined as the monetary measure of all finished goods and services in a country within a specific period
 - If you don't do a careful analysis of each attribute and its relation to GDP, you might let a leakage happen:
 - two columns, Population and GDP per capita, multiplied, equal GDP
 - The model will perfectly predict GDP by looking at these two columns only
 - The fact that GDP be one of the features, though in a slightly modified form (devised by the population), constitutes contamination and, therefore, leads to data leakage
- Imagine training a model to predict the yearly salary, given the attributes of an employee
 - The training data is a table that contains both monthly and yearly salary, among many other attributes.
 - If monthly salary is not removed from the list of features, that attribute alone will perfectly predict the yearly salary,
 - making believe that model is perfect
 - Once the model is put in production, it will like about a person's monthly salary: otherwise, the modeling would not be needed

Country	Population	Region	...	GDP per capita	GDP
France	67M	Europe	...	38,800	2.6T
Germany	83M	Europe	...	44,578	3.7T
...	...	...	...	...	...
China	1386M	Asia	...	8,802	12.2T

An example of the target (GDP) being a simple function of two features:
Population and GDP per capita.

Sometimes the target is not a function of one or more features, but rather is “hidden” in one of the features.

Using customer data to predict their gender

Customer ID	Group	Yearly Spendings	Yearly Pageviews	...	Gender
1	M18-25	1350	11,987	...	M
2	F25-35	2365	8,543	...	F
...	...	...	...	...	...
18879	F65+	3653	6,775	...	F

An example of the target being hidden in one of the features.

- Look at the column Group
- If you closely investigate the data in the column Group, represents a demographic value to which each existing customer was related in the past.
- If the data about a customer’s gender and age is factual, then the column Group constitutes a form of data leakage
 - the value want to predict is “hidden” in the value of a feature
- On the other hand, if the Group values are predictions provided by another, possibly less accurate model
 - then it can be used to build a potentially stronger model - model stacking.



Feature coming from the future

Feature from the future is a kind of data leakage that is hard to catch if don't have a clear understanding of the business goal

- Imagine a client asked to train a model that predicts whether a borrower will pay back the loan based on attributes such as age, gender, education, salary, marital status, and so on
 - might decide to use all available attributes to predict the value in the column Will Pay Loan, including the data from the column Late Payment Reminders.
 - Model will look accurate at testing time and model doesn't work well in the production environment in the production environment. the value of Late Payment Reminders is always zero
 - makes sense because the client uses model before the borrower gets the credit, so no reminders have yet been n
 - However, model most likely learned to mal
- Late Payment Reminders is 1 or more and pays less attention to the other features
- Understanding the business context in which the model will be used is, thus, crucial to avoid data leakage.

Borrower ID	Demographic Group	Education	...	Late Payment Reminders	Will Pay Loan
1	M35-50	High school	...	0	Y
2	F25-35	Master's	...	1	N
...	...	...	...	...	...
65723	M25-35	Master's	...	3	N

A feature unavailable at the prediction time: Late Payment Reminders.

- Evaluating a machine learning model responsibly requires doing more than just calculating loss metrics
 - critical to audit training data and evaluate predictions for bias
- Machine learning models are not inherently objective
 - Engineers train models by feeding them a data set of training examples
 - This human involvement in the provision and curation of this data can make a model's predictions susceptible to bias.
- Important to be aware of common human biases that can manifest in data
 - proactive steps can be taken to mitigate their effects



- Occurs when the frequency of events, properties, and/or outcomes captured in a data set does not accurately reflect their real-world frequency
- Can arise because people tend to focus on documenting circumstances that are unusual or especially memorable
- EXAMPLE:
- A sentiment-analysis model is trained to predict whether book reviews are positive or negative based on a corpus of user submissions to a popular website. The majority of reviews in the training data set reflect extreme opinions (reviewers who either loved or hated a book), because people were less likely to submit a review of a book if they did not respond to it strongly. As a result, the model is less able to correctly predict sentiment of reviews that use more subtle language to describe a book.



- Tendency to favor results generated by automated systems over those generated by non-automated systems, irrespective of the error rates of each

EXAMPLE:

- Software engineers working for a sprocket manufacturer were eager to deploy the new "groundbreaking" model they trained to identify tooth defects, until the factory supervisor pointed out that the model's precision and recall rates were both 15% lower than those of human inspectors.



- Selection bias occurs if a data set's examples are chosen in a way that is not reflective of their real-world distribution
- Coverage bias: Data is not selected in a representative fashion.
EXAMPLE: A model is trained to predict future sales of a new product based on phone surveys conducted with a sample of consumers who bought the product. Consumers who instead opted to buy a competing product were not surveyed, and as a result, this group of people was not represented in the training data.
- Non-response bias (or participation bias): Data ends up being unrepresentative due to participation gaps in the data-collection process.
 - EXAMPLE: A model is trained to predict future sales of a new product based on phone surveys conducted with a sample of consumers who bought the product and with a sample of consumers who bought a competing product. Consumers who bought the competing product were 80% more likely to respond to the survey.



Types of Bias - Group Attribution Bias

- Tendency to generalize what is true of individuals to an entire group to which they belong
- Two key manifestations are:

In-group bias

- A preference for members of a group to which you also belong, or for characteristics that you also share.
- EXAMPLE: Two engineers training a résumé-screening model for software developers are predisposed to believe that applicants who attended the same computer-science academy as they both did are more qualified for the role.

Out-group homogeneity bias

- A tendency to stereotype individual members of a group to which you do not belong, or to see their characteristics as more uniform.
- EXAMPLE: Two engineers training a résumé-screening model for software developers are predisposed to believe that all applicants who did not attend a computer-science academy do not have sufficient expertise for the role.

Types of Bias - Implicit Bias



- Occurs when assumptions are made based on one's own mental models and personal experiences that do not necessarily apply more generally
 - EXAMPLE:
 - An engineer training a gesture-recognition model uses a head shake as a feature to indicate a person is communicating the word "no." However, in some regions of the world, a head shake actually signifies "yes."
 - Confirmation bias
 - Model builders unconsciously process data in ways that affirm preexisting beliefs and hypotheses
 - Experimenter's bias
 - In some cases, a model builder may actually keep training a model until it produces a result that aligns with their original hypothesis
- EXAMPLE:
- An engineer is building a model that predicts aggressiveness in dogs based on a variety of features (height, weight, breed, environment). The engineer had an unpleasant encounter with a hyperactive toy poodle as a child, and ever since has associated the breed with aggression. When the trained model predicted most toy poodles to be relatively docile, the engineer retrained the model several more times until it produced a result showing smaller poodles to be more violent.



If your data set has one or more features that have missing values for a large number of examples, that could be an indicator that certain key characteristics of your data set are under-represented.

For example, the table below shows a summary of key stats for a subset of features in the California Housing dataset

Suppose instead that three features (population, households, and median_income) only had a count of 3000-in other words, that there were 14,000 missing values for each feature.

	longitude	latitude	total_rooms	population	households	median_income	median_house_value
count	17000.0	17000.0	17000.0	3000.0	3000.0	3000.0	17000.0
mean	-119.6	35.6	2643.7	1429.6	501.2	3.9	207.3
std	2.0	2.1	2179.9	1147.9	384.5	1.9	116.0
min	-124.3	32.5	2.0	3.0	1.0	0.5	15.0
25%	-121.8	33.9	1462.0	790.0	282.0	2.6	119.4
50%	-118.5	34.2	2127.0	1167.0	409.0	3.5	180.4
75%	-118.0	37.7	3151.2	1721.0	605.2	4.8	265.0
max	-114.3	42.0	37937.0	35682.0	6082.0	15.0	500.0

These 14,000 missing values would make it much more difficult to accurately correlate average income of households with median house prices.

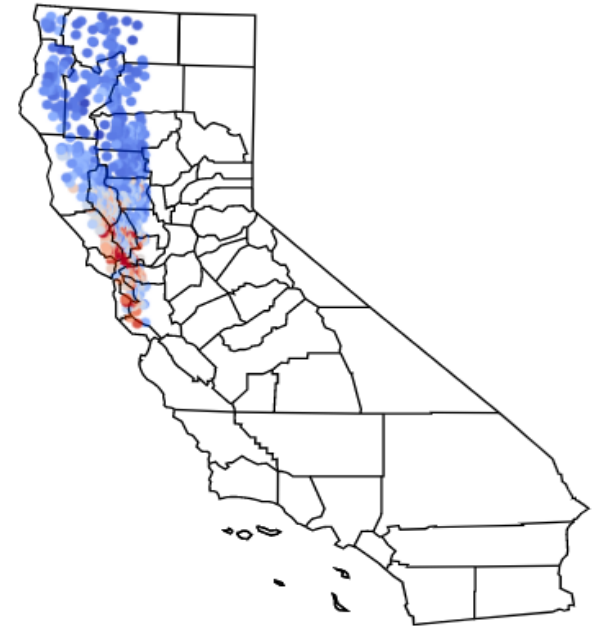
Before training a model on this data, it would be prudent to investigate the cause of these missing values to ensure that there are no latent biases responsible for missing income and population data.



- When exploring data, you should also look for examples that contain feature values that stand out as especially uncharacteristic or unusual
- These unexpected feature values could indicate problems that occurred during data collection or other inaccuracies that could introduce bias.
- Can you pinpoint any unexpected feature values?

	longitude	latitude	total_rooms	population	households	median_income	median_house_value
1	-121.7	38.0	7105.0	3523.0	1088.0	5.0	0.2
2	-122.4	37.8	2479.0	1816.0	496.0	3.1	0.3
3	-122.0	37.0	2813.0	1337.0	477.0	3.7	0.3
4	-103.5	43.8	2212.0	803.0	144.0	5.3	0.2
5	-117.1	32.8	2963.0	1162.0	556.0	3.6	0.2
6	-118.0	33.7	3396.0	1542.0	472.0	7.4	0.4

- Any sort of skew in your data, where certain groups or characteristics may be under- or over-represented relative to their real-world prevalence, can introduce bias into your model.
- Figure visualizes a subset of data drawn from the full housing data set that exclusively represents the northwest region of California
- If this unrepresentative sample were used to train a model to predict California housing prices statewide, the lack of housing data from southern portions of California would be problematic
- The geographical bias encoded in the model might adversely affect homebuyers in unrepresented communities.





- In many large-scale solutions, data is divided into **partitions** can be managed and accessed separately
- Partitioning can
 - improve scalability,
 - reduce contention,
 - and optimize performance
- Can also provide a mechanism for **dividing data by usage pattern**
 - can archive older data in cheaper data storage
- However, the partitioning strategy must be chosen carefully to maximize the benefits while minimizing adverse effects



Reasons • Improve scalability

- When a single database system scaled up, it will eventually reach a physical hardware limit
- If **data is divided across multiple partitions**, each hosted on a separate server, can **scale out** the system almost **indefinitely**

• Improve performance

- **Data access operations on each partition take place over a smaller volume of data**
- Correctly done, partitioning can make system more efficient
- Operations that affect more than one partition **can run in parallel**

• Improve availability

- Separating data across multiple servers **avoids a single point of failure**
- If one instance fails, only the data in that partition is unavailable, operations on other partitions can continue

• Improve security

- In some cases, **sensitive and non-sensitive data can be separated** into different partitions and apply different security controls to the sensitive data

Three typical strategies for partitioning data

- Horizontal partitioning (aka sharding)
 - each partition is a separate data store, but all partitions have the same schema
 - each partition is known as a shard and holds a specific subset of the data
 - such as all the orders for a specific set of customers
- Vertical partitioning
 - each partition holds a subset of the fields for items in the data store
 - fields are divided according to their pattern of use
 - frequently accessed fields might be placed in one vertical partition and less frequently accessed fields in another
- Functional partitioning
 - data is aggregated according to how it is used by each bounded context in the system
 - an e-commerce system might store invoice data in one partition and product inventory data in another
- These strategies can be combined
 - might divide data into shards and then use vertical partitioning to further subdivide the data in each shard

Example Product data

- Product inventory data is **divided** into shards based on **the product key**
- Each shard holds the data for a contiguous range of shard keys (A-G and H-Z) organized alphabetically
- Sharding spreads the load over more computers, which reduces contention and improves performance.

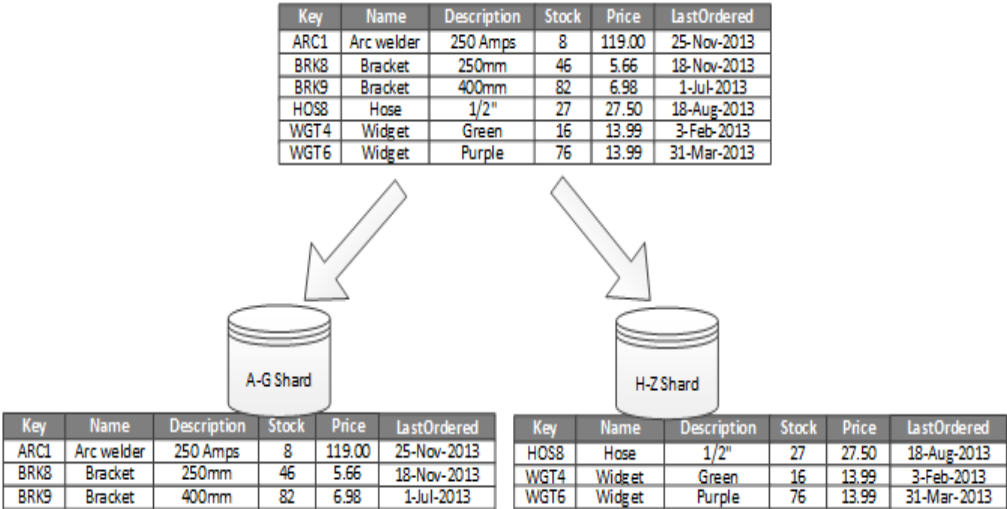


Figure - Horizontally partitioning (sharding) data based on a partition key.

Horizontal partitioning (sharding) – 2 - Practices



- **Sharding key**
 - most important factor is the **choice of a sharding key**
 - can be **difficult to change the key** after the system is in operation
 - key must ensure that **data is partitioned to spread the workload as evenly as possible across the shards**
 - Choose a sharding key that minimizes any future requirements to split large shards, coalesce small shards into larger partitions,
 - Changing the schema can be **very time consuming**, and might require taking one or more **shards offline** while they are performed
- **Size**
 - The **shards don't have to be the same size**
 - more important to **balance the number of requests**
 - Some shards might be very large, but each item has a low number of access operations
 - Other shards might be smaller, but each item is accessed much more frequently
- **Access**
 - **Avoid creating "hot" partitions** that can **affect performance and availability**
 - using the first letter of a customer's name causes an unbalanced distribution, because some letters are more common
 - instead, use a hash of a customer identifier to distribute data more evenly across partitions
- **Replication**
 - If shards are replicated, it might be **possible to keep some of the replicas online while others are split, merged, or reconfigured**
 - the system might need to limit the operations that can be performed during the reconfiguration
 - the data in the replicas might be marked as read-only to prevent data inconsistencies

Example of vertical partitioning

- Different properties of an item are stored in different partitions
 - One partition holds data that is accessed more frequently, including product name, description, and price
 - Another partition holds inventory data: the stock count and last-ordered date

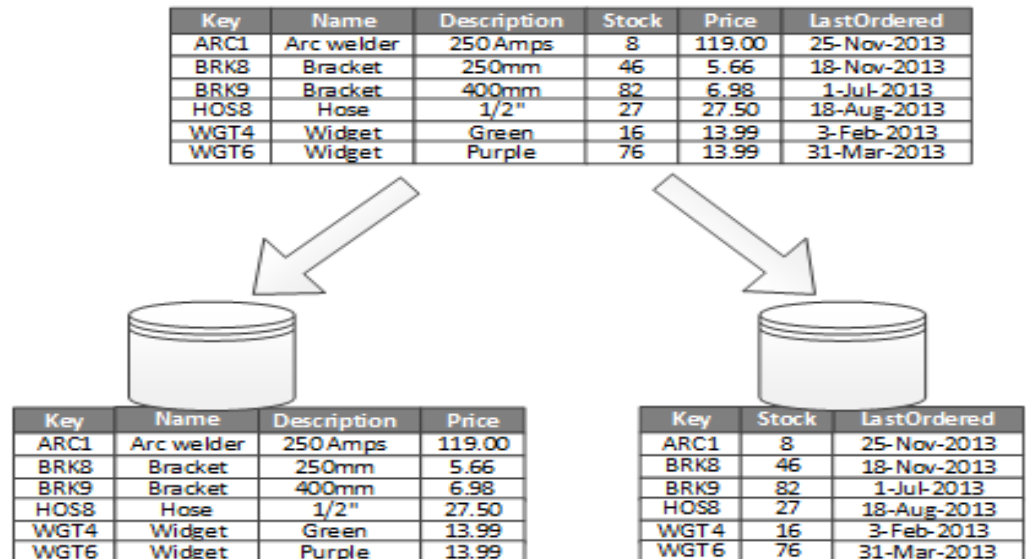
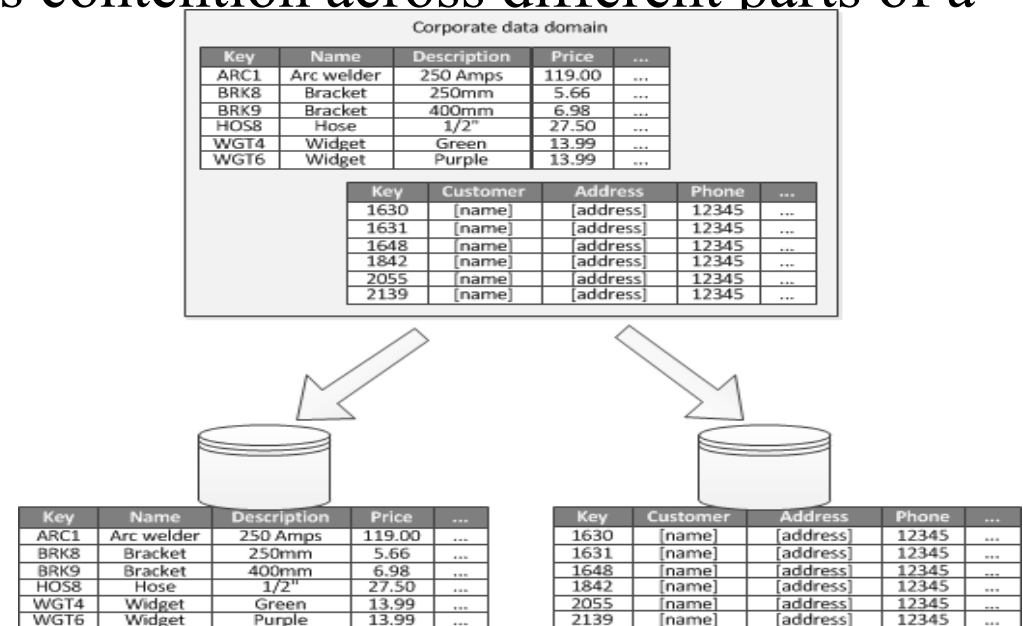


Figure . - Vertically partitioning data by its pattern of use.



- The most common use is to reduce the I/O and performance costs associated with fetching items that are frequently accessed
- Vertical partitioning operates at the entity level within a data store,
 - partially normalizing an entity to break it down from a wide item to a set of narrow items
 - ideally suited for column-oriented data stores such as HBase and Cassandra
- Relatively slow-moving data (product name, description, and price) can be separated from the more dynamic data (stock level and last ordered date)
 - Slow moving data is a good candidate for an application to cache in memory
- Sensitive data can be stored in a separate partition with additional security controls

- Functional partitioning is a way to improve isolation and data access performance
 - suitable when possible to identify a bounded context for each distinct business area in an application
 - can be used to separate read-write data from read-only data
 - can help reduce data access contention across different parts of a system.



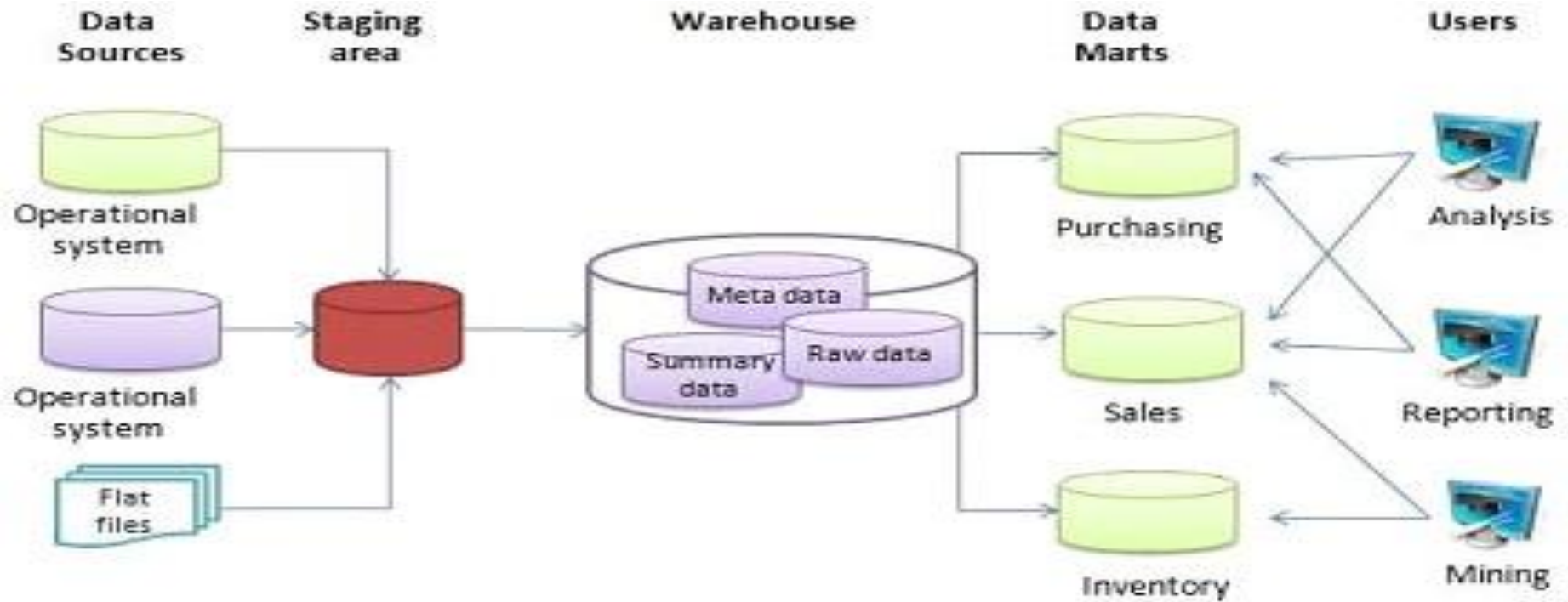
Functionally partitioning data by bounded context or subdomain.

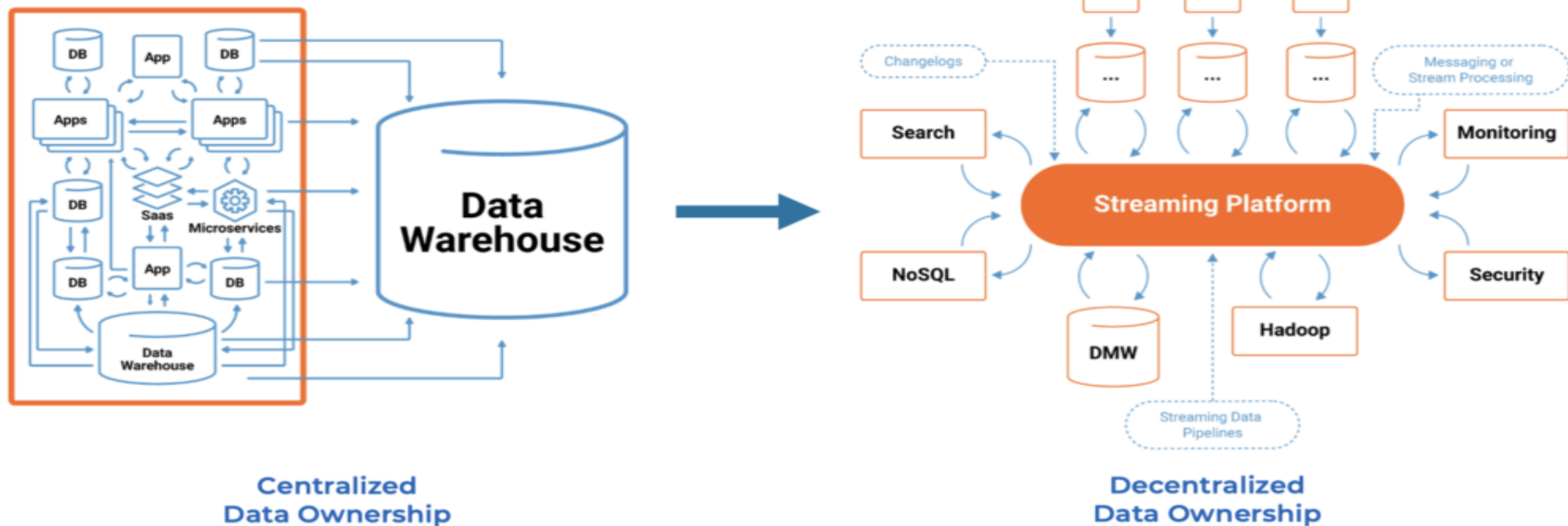


- As a system matures, need to adjust the partitioning scheme!
- For example,
 - individual partitions might start getting a disproportionate volume of traffic and become hot, leading to excessive contention
 - might have underestimated the volume of data in some partitions, causing some partitions to approach capacity limits
- Some data stores, such as Azure Cosmos DB, can **automatically rebalance partitions**
- In other cases, **rebalancing is an administrative task** that consists of two stages:
 - Determine a new partitioning strategy.
 - Which partitions need to be split (or possibly combined)?
 - What is the new partition key?
 - Migrate data from the old partitioning scheme to the new set of partitions.
- Migrations
 - Depending on the data store, might be able to **migrate data between partitions while they are in use - online migration**
 - If that's not possible, might **need to make partitions unavailable while the data is relocated - offline migration**

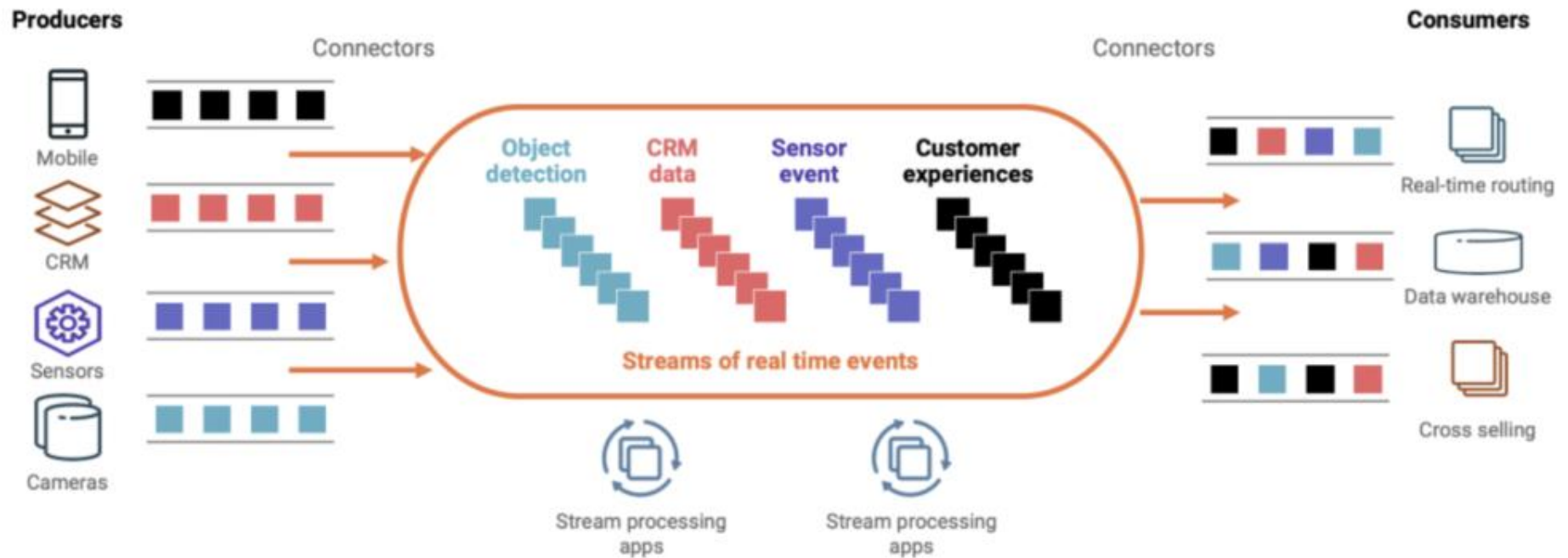


- **Offline migration**
 - is typically simpler because it reduces the chances of contention occurring
 - Works as follows:
 - Mark the partition offline
 - Split-merge and move the data to the new partitions
 - Verify the data
 - Bring the new partitions online
 - Remove the old partition
 - Optionally, you can mark a partition as read-only in step 1, so that applications can still read the data while it is being moved
- **Online migration**
 - more complex to perform but less disruptive
 - similar to offline migration, except the original partition is not marked offline
 - Depending on the granularity of the migration process, the data access code in the client applications
 - might have to handle reading and writing data that's held in two locations, the original partition and the new partition





- A data warehouse is excellent for **reporting and business intelligence**.
- A data lake is perfect for **big data analytics and AI / Machine Learning**.
- **Data streaming** enables **real-time use cases**.
- A **decentralized, flexible enterprise architecture** is required to build a modern data stack around **microservices and data mesh**.



Use Cases for Data Streaming by Business Value

